

01B JS Primitives Paradox

JavaScript Primitives

1. Introduction to Primitives

Definition: A primitive is a value of a primitive type - the most basic, fundamental data type in JavaScript.

Key Characteristic: Primitives represent a *single value*, not a collection of values.

2. The Seven Primitive Types

JavaScript has exactly **7 primitive types**:

1. **string** - Text data (`"hello"`, `'world'`)
 2. **number** - Numeric values (`42`, `3.14`, `NaN`, `Infinity`)
 3. **bigint** - Large integers beyond `Number` limits (`123n`)
 4. **boolean** - Logical values (`true`, `false`)
 5. **symbol** - Unique identifiers (created with `Symbol()`)
 6. **null** - Intentional absence of value
 7. **undefined** - Variable declared but not assigned
-

3. Primitives vs Objects: Core Differences

Primitives:

- Store a **single value**
- **Immutable** - cannot be changed
- **Lightweight** - minimal memory overhead
- **Fast** - optimized by JavaScript engine
- Stored directly in memory (typically on the stack)

Objects:

- Store **multiple values** as properties
- **Mutable** - can be modified
- **"Heavier"** - require additional resources for internal machinery
- Created with `{}`, arrays `[]`, functions, dates, etc.
- Stored by reference (typically on the heap)

```
// Primitive
let x = 5;
let y = x; // y gets a COPY of the value
x = 10;
console.log(y); // 5 (unchanged)

// Object
let obj1 = { value: 5 };
let obj2 = obj1; // obj2 gets a REFERENCE
obj1.value = 10;
console.log(obj2.value); // 10 (changed!)
```

4. The Paradox: Primitives with Methods

The Problem:

- Primitives should be lightweight and simple
- But we want convenient methods like `str.toUpperCase()` or `num.toFixed(2)`
- How can primitives have methods if they're not objects?

The Solution: Temporary Object Wrappers

JavaScript uses a clever mechanism:

1. **Primitives remain primitive** - lightweight and fast
2. **When you access a method/property**, JavaScript temporarily creates a "wrapper object"
3. **The wrapper provides the functionality**
4. **The wrapper is immediately destroyed** after use

5. Object Wrappers Explained

The Five Wrapper Types:

- `String` - wraps string primitives
- `Number` - wraps number primitives
- `Boolean` - wraps boolean primitives
- `Symbol` - wraps symbol primitives
- `BigInt` - wraps bigint primitives

How It Works (Step-by-Step) :

```
let str = "hello";
alert(str.toUpperCase()); // "HELLO"
```

Behind the scenes:

1. `str` is a primitive string
2. When accessing `.toUpperCase()`, JavaScript creates: `new String("hello")`
3. The method runs on this temporary object
4. The result (`"HELLO"`) is returned
5. The temporary `String` object is destroyed
6. `str` remains a primitive

Visual Representation:

```
str (primitive "hello")
↓
Access .toUpperCase()
↓
Create temporary: new String("hello")
↓
Call method: temporary.toUpperCase()
↓
Return result: "HELLO"
↓
Destroy temporary object
↓
str still primitive "hello"
```

6. Important Examples

Example 1: Using Primitive Methods

```
// String methods
let text = "JavaScript";
console.log(text.toLowerCase()); // "javascript"
console.log(text.charAt(0));      // "J"
console.log(text.slice(0, 4));    // "Java"

// Number methods
let num = 3.14159;
console.log(num.toFixed(2));      // "3.14"
console.log(num.toString());      // "3.14159"

// Boolean - fewer methods, but still:
let bool = true;
console.log(bool.toString());     // "true"
```

JS

Example 2: Why Primitives Can't Store Properties

```
let str = "Hello";  
str.test = 5;           // Attempt to add property  
  
console.log(str.test); // undefined
```

JS

What happens:

1. Line 2: `str.test = 5`

- Creates temporary `String` object
- Sets `test` property on temporary object
- Temporary object destroyed (property lost!)

2. Line 4: `console.log(str.test)`

- Creates a *new* temporary `String` object
- This new object doesn't have `test` property
- Returns `undefined`

In strict mode:

```
"use strict";  
let str = "Hello";  
str.test = 5; // TypeError: Cannot create property 'test' on string 'Hello'
```

JS

7. Constructor Functions: A Warning

⚠ DON'T DO THIS:

```
let num = new Number(42);  
let str = new String("hello");  
let bool = new Boolean(false);
```

JS

Why this is bad:

```
// typeof returns "object" not the primitive type
typeof new Number(42);    // "object" ✗
typeof 42;                // "number" ✓

// Objects are always truthy, even Boolean(false)!
let falsyObject = new Boolean(false);
if (falsyObject) {
  console.log("This runs!"); // Confusing! ✗
}

// Direct boolean works correctly
let falsyPrimitive = false;
if (falsyPrimitive) {
  console.log("This doesn't run"); // Expected ✓
}
```

✓ DO THIS INSTEAD:

Use wrapper functions **without** `new` for type conversion:

```
let num = Number("123");    // Converts to primitive number
let str = String(123);      // Converts to primitive string
let bool = Boolean(0);      // Converts to primitive boolean

typeof num;    // "number" ✓
typeof str;    // "string" ✓
typeof bool;   // "boolean" ✓
```

8. Special Cases: null and undefined

The Exceptions:

- `null` and `undefined` are primitives
- They have **NO wrapper objects**
- They provide **NO methods**
- They are "the most primitive"

JS

```
null.toString();           // TypeError: Cannot read property 'toString' of null
undefined.toString();      // TypeError: Cannot read property 'toString' of undefined

// Other primitives work fine:
"hello".toString();        // "hello" ✓
(42).toString();           // "42" ✓
true.toString();           // "true" ✓
```

9. Performance Considerations

JavaScript Engine Optimizations:

- The engine is **highly optimized** for this wrapper mechanism
- May skip creating wrapper objects entirely in some cases
- Must still **behave** as if wrappers are created (per specification)
- Calling primitive methods is **not expensive**

JS

```
// This looks like it creates objects, but it's optimized:
for (let i = 0; i < 1000000; i++) {
  "hello".toUpperCase(); // Very fast despite "object creation"
}
```

10. Practical Implications

Immutability:

JS

```
let str = "hello";
str.toUpperCase(); // Returns "HELLO"
console.log(str);  // Still "hello" - primitives are immutable!

// Must reassign to change:
str = str.toUpperCase();
console.log(str);   // Now "HELLO"
```

Comparison:

```
// Primitives compare by value
let a = "hello";
let b = "hello";
console.log(a === b);    // true ✓

// Objects compare by reference
let obj1 = { value: "hello" };
let obj2 = { value: "hello" };
console.log(obj1 === obj2); // false ✗
```

JS

11. Summary

Key Takeaways:

1. **Primitives are simple values:** string, number, bigint, boolean, symbol, null, undefined
2. **Primitives are immutable and lightweight** - this is their strength
3. **Temporary wrappers enable methods** - best of both worlds:
 - Primitives stay fast and simple
 - We still get convenient methods
4. **Wrappers are automatic and temporary** - created and destroyed automatically
5. **Cannot store properties on primitives** - wrappers are discarded immediately
6. **Never use `new` with String/Number/Boolean** - creates confusing object types
7. **null and undefined have no methods** - attempting to call methods throws errors
8. **The mechanism is highly optimized** - don't worry about performance

12. Common Pitfalls

JS

```
// ❌ Pitfall 1: Trying to store data on primitives
let str = "test";
str.customProperty = 100;
console.log(str.customProperty); // undefined

// ❌ Pitfall 2: Using new with wrapper constructors
let num = new Number(0);
if (num) { /* Always runs - objects are truthy! */ }

// ❌ Pitfall 3: Forgetting primitives are immutable
let text = "hello";
text.toUpperCase();
console.log(text); // Still "hello" - need to reassign!

// ✅ Correct approach:
text = text.toUpperCase();
console.log(text); // "HELLO"
```

This elegant design allows JavaScript to maintain the simplicity and performance of primitives while providing the convenience of object methods - a perfect balance between efficiency and developer experience.